
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 8

Caracteres e Strings em C

Fazendo a Diferença

Exercícios 8.43 a 8.46

Receitas saudáveis · Detector de spam
Linguagem SMS · Neutralidade de gênero

Contents

1	Exercício 8.43 – Cozinhando com Ingredientes Saudáveis	2
2	Exercício 8.44 – Analisador de Spam	5
3	Exercício 8.45 – Linguagem SMS	8
4	Exercício 8.46 – Neutralidade de Gênero	10

1. Exercício 8.43 – Cozinhando com Ingredientes Saudáveis

Enunciado 8.43

Escreva um programa que leia uma receita do usuário e sugira substitutos mais saudáveis para ingredientes. O programa deve considerar problemas como colesterol alto, perda de peso e alergias. Use as substituições típicas da tabela do livro (Figura 8.40).

Pesquisa: Substituições de Ingredientes

Pesquisa externa realizada

Conforme a tabela da Figura 8.40 do livro (com fontes em sites de nutrição):

- 1 xícara de creme de leite → 1 xícara de iogurte
- 1 xícara de açúcar → 1/2 xícara de mel ou 1/4 de adoçante
- 1 xícara de manteiga → 1 xícara de margarina/iogurte
- 1 ovo → 2 colheres de amido + 2 claras (ou banana amassada)
- Farinha branca → farinha integral ou de arroz
- 1 xícara de maionese → 1 xícara de cottage

A lógica do programa: percorrer a receita, identificar ingredientes-alvo via `strstr`, e substituí-los conforme o perfil de saúde do usuário.

Solução em C

Resposta detalhada

```
1  /* Ex8.43: receita_saudavel.c
2     Sugere substituições mais saudáveis em receitas */
3  #include <stdio.h>
4  #include <string.h>
5  #include <ctype.h>
6  #define MAX_RECEITA 2000
7
8  /* Estrutura de substituição */
9  typedef struct {
10     const char *original;
11     const char *substituto;
12     const char *motivo; /* "colesterol", "perda peso", "alergia" */
13 } Substituicao;
14
15 void sugerir( const char *receita, const Substituicao
16             subs[], int n );
17
18 int main( void )
19 {
20     Substituicao subs[] = {
21         { "creme de leite", "iogurte natural",
```

```

21         "colesterol/peso" },
22     { "manteiga",          "azeite ou margarina light",
23       "colesterol" },
24     { "acucar",           "mel ou adocante",
25       "peso/diabetes" },
26     { "maionese",         "queijo cottage",
27       "colesterol" },
28     { "ovo",              "2 claras + 2 colheres
29       amido","colesterol" },
30     { "farinha branca", "farinha integral",
31       "perda peso" },
32     { "leite",            "leite desnatado/soja",
33       "colesterol" }
34 };
35 int n = sizeof( subs ) / sizeof( subs[ 0 ] );
36
37 char receita[ MAX_RECEITA ];
38
39 printf( "==== CONVERSOR DE RECEITAS SAUAVEIS
40 =====\n\n" );
41 printf( "ATENCAO: Sempre consulte um medico antes
42 de fazer\n" );
43 printf( "          mudancas significativas em sua
44 dieta.\n\n" );
45
46 printf( "Digite sua receita (digite END em uma
47 linha sozinha):\n" );
48
49 /* Le linhas ate encontrar "END" */
50 receita[ 0 ] = '\0';
51 char linha[ 200 ];
52 while ( fgets( linha, sizeof( linha ), stdin ) !=
53         NULL ) {
54     if ( strncmp( linha, "END", 3 ) == 0 ) break;
55     strcat( receita, linha );
56 }
57
58 sugerir( receita, subs, n );
59
60 return 0;
61 }
62
63 void sugerir( const char *receita, const Substituicao
64 subs[], int n )
65 {
66     int i;
67     int encontrou_algo = 0;
68
69     printf( "\n===== SUGESTOES =====\n" );
70
71

```

```
58     for ( i = 0; i < n; ++i ) {
59         if ( strstr( receita, subs[ i ].original ) !=
60             NULL ) {
61             printf( "- Substitua \"%s\" por \"%s\"\n",
62                     subs[ i ].original, subs[ i ].
63                     substituto );
64             printf( "  (beneficio: %s)\n", subs[ i ].
65                     motivo );
66             encontrou_algo = 1;
67         }
68     }
69
70     if ( !encontrou_algo ) {
71         printf( "Nenhuma substituicao aplicavel
72                 encontrada.\n" );
73         printf( "Sua receita parece estar com
74                 ingredientes ja saudaveis!\n" );
75     }
76 }
```

2. Exercício 8.44 – Analisador de Spam

Enunciado 8.44

Pesquise palavras comuns em spam e crie uma lista com 30. Escreva um programa que leia uma mensagem (sem ultrapassar o limite do array), conte ocorrências dessas palavras-chave, e avalie a probabilidade de a mensagem ser spam.

Solução em C

Resposta detalhada

```
1  /* Ex8.44: analisador_spam.c */
2  #include <stdio.h>
3  #include <string.h>
4  #include <ctype.h>
5  #define MAX_MSG 5000
6  #define NUM_PALAVRAS 30
7
8  int main( void )
9  {
10     const char *palavrasSpam[ NUM_PALAVRAS ] = {
11         "gratis",      "promocao",    "ganhe",      "
12         premio",
13         "credito",     "oferta",      "dinheiro",   "
14         urgente",
15         "exclusivo",   "limitado",   "100%",       "
16         garantido",
17         "renda",       "milionario", "investimento",
18         "lucro",
19         "viagra",      "remedio",    "farmacia",   "
20         barato",
21         "desconto",    "imperdivel", "milagre",     "
22         secreto",
23         "rapido",      "sucesso",    "saude",       "
24         emagrecer",
25         "clique aqui", "ganhador"
26     };
27
28     char msg[ MAX_MSG ];
29     char msgLower[ MAX_MSG ];
30     int pontos = 0;
31     int i, j;
32     char linha[ 200 ];
33
34     printf( "Digite a mensagem de e-mail (END para
35             terminar):\n" );
36
37     msg[ 0 ] = '\0';
```

```
30 while ( fgets( linha, sizeof( linha ), stdin ) !=  
31         NULL ) {  
32     if ( strncmp( linha, "END", 3 ) == 0 ) break;  
33  
34     /* Garante que nao excede o tamanho */  
35     if ( strlen( msg ) + strlen( linha ) < MAX_MSG  
36         - 1 ) {  
37         strcat( msg, linha );  
38     }  
39  
40     /* Converte para minusculas para busca case-  
41        insensitive */  
42     for ( i = 0; msg[ i ] != '\0'; ++i ) {  
43         msgLower[ i ] = tolower( ( unsigned char )msg[  
44             i ] );  
45     }  
46     msgLower[ i ] = '\0';  
47  
48     /* Conta ocorrencias de cada palavra-chave */  
49     printf( "\n=== ANALISE ===\n" );  
50     for ( i = 0; i < NUM_PALAVRAS; ++i ) {  
51         int conta = 0;  
52         char *p = msgLower;  
53         while ( ( p = strstr( p, palavrasSpam[ i ] ) )  
54             != NULL ) {  
55             ++conta;  
56             ++p;  
57         }  
58         if ( conta > 0 ) {  
59             printf( " '%s' aparece %d vez(es)\n",  
60                 palavrasSpam[ i ], conta );  
61             pontos += conta;  
62         }  
63     }  
64  
65     /* Avaliacao */  
66     printf( "\nNivel de spam: %d pontos\n", pontos );  
67     if ( pontos == 0 ) {  
68         printf( "Provavelmente NAO e spam.\n" );  
69     }  
70     else if ( pontos <= 2 ) {  
71         printf( "Possivelmente legitimo.\n" );  
72     }  
73     else if ( pontos <= 5 ) {  
74         printf( "SUSPEITO: pode ser spam.\n" );  
75     }  
76     else {  
77         printf( "MUITO PROVAVELMENTE SPAM!\n" );  
78     }
```

```
74  
75     return 0;  
76 }
```

Reflexão

Boa prática de programação

Filtros de spam reais usam técnicas muito mais sofisticadas:

- **Bayes ingênuo:** cálculo probabilístico baseado em palavras e suas frequências em corpus de spam vs. ham.
- **Análise de cabeçalhos:** verificação de remetente, SPF, DKIM.
- **Listas de bloqueio (blacklists):** domínios e IPs conhecidos por enviar spam.
- **Aprendizado de máquina:** redes neurais treinadas em milhões de mensagens.

O exercício é uma porta de entrada para esse domínio fascinante, em que estatística e linguística computacional encontram a engenharia de software.

3. Exercício 8.45 – Linguagem SMS

Enunciado 8.45

Escreva um programa que traduza mensagens em *linguagem SMS* (abreviações de mensagens de texto) para português padrão e vice-versa. Lembre-se de que algumas abreviações podem ter múltiplos significados (ambiguidade).

Solução em C

Resposta detalhada

```
1  /* Ex8.45: tradutor_sms.c */
2  #include <stdio.h>
3  #include <string.h>
4  #include <ctype.h>
5  #define MAX_MSG 1000
6
7  typedef struct {
8      const char *sms;
9      const char *portugues;
10 } Abreviacao;
11
12 void substituirTodas( char *texto, const char *de,
13                     const char *para );
14
15 int main( void )
16 {
17     Abreviacao tabela[] = {
18         { "vc", "voce" }, { "tb", "tambem" },
19         { "blz", "beleza" }, { "pq", "porque" },
20         { "tbm", "tambem" }, { "qq", "qualquer" },
21         { "qdo", "quando" }, { "msm", "mesmo" },
22         { "td", "tudo" }, { "qto", "quanto" },
23         { "obg", "obrigado" }, { "vlw", "valeu" },
24         { "abs", "abraços" }, { "bjs", "beijos" },
25         { "emo", "em minha opiniao" }
26     };
27     int n = sizeof( tabela ) / sizeof( tabela[ 0 ] );
28
29     int opcao;
30     char texto[ MAX_MSG ];
31     int i;
32
33     printf( "1 - SMS -> Portugues\n2 - Portugues -> SMS\nEscolha: " );
34     scanf( "%d", &opcao );
35     getchar(); /* consome \n */
```

```
36     printf( "Digite a mensagem: " );
37     fgets( texto, MAX_MSG, stdin );
38
39     if ( opcao == 1 ) {
40         for ( i = 0; i < n; ++i ) {
41             substituirTodas( texto, tabela[ i ].sms,
42                             tabela[ i ].portugues );
43         }
44     }
45     else {
46         for ( i = 0; i < n; ++i ) {
47             substituirTodas( texto, tabela[ i ].
48                             portugues, tabela[ i ].sms );
49         }
50     }
51
52     printf( "\nResultado: %s\n", texto );
53     return 0;
54 }
55
56 void substituirTodas( char *texto, const char *de,
57                     const char *para )
58 {
59     char buffer[ MAX_MSG ];
60     char *p;
61     int tamDe = strlen( de );
62     int tamPara = strlen( para );
63
64     while ( ( p = strstr( texto, de ) ) != NULL ) {
65         /* Cópia ate o ponto encontrado */
66         int prefixo = p - texto;
67         strncpy( buffer, texto, prefixo );
68         buffer[ prefixo ] = '\0';
69
70         /* Concatena substituição */
71         strcat( buffer, para );
72
73         /* Concatena resto apos o termo encontrado */
74         strcat( buffer, p + tamDe );
75
76         /* Cópia de volta */
77         strcpy( texto, buffer );
78     }
79 }
```

Problema da ambiguidade: “EMO” pode significar “em minha opinião”, “Empresa Marítima Ocidental”, ou um gênero musical. Sistemas reais usam contexto para resolver: vizinhança de outras palavras, modelos estatísticos de linguagem, ou perguntar ao usuário. Esse é um problema clássico de *desambiguação lexical*.

4. Exercício 8.46 – Neutralidade de Gênero

Enunciado 8.46

Crie um programa que leia um parágrafo de texto e substitua palavras de gênero específico por equivalentes sem gênero. Apresente o texto resultante.

Solução em C

Resposta detalhada

```
1  /* Ex8.46: neutralizar_genero.c */
2  #include <stdio.h>
3  #include <string.h>
4  #include <ctype.h>
5  #define MAX_TEXTO 5000
6
7  typedef struct {
8      const char *generificado;
9      const char *neutro;
10 } Substituicao;
11
12 void substituirTodas( char *texto, const char *de,
13                     const char *para );
14
15 int main( void )
16 {
17     /* Tabela de substituicoes para neutralidade de
18        genero
19        (apenas alguns exemplos -- expandir conforme a
20        necessidade) */
21     Substituicao subs[] = {
22         { "porta-voz", "representante" },
23         { "presidente", "presidencia" },
24         { "homem-feito", "industrial" },
25         { "homens", "pessoas" },
26         { "homem", "pessoa" },
27         { "policial", "policial" }, /*
28            ja neutro */
29         { "bombeiro", "agente do corpo de bombeiros" },
30         { "garcom", "atendente" },
31         { "carteiro", "agente postal" },
32         { "merceeiro", "comerciante" },
33         { "marinheiro", "tripulante" },
34         { "doutora", "doutor(a)" }
35     };
36
37     int n = sizeof( subs ) / sizeof( subs[ 0 ] );
38
39     char texto[ MAX_TEXTO ];
```

```
35     char linha[ 500 ];
36     int i;
37
38     printf( "Digite o paragrafo (END para terminar):\n"
39            " );
40
41     texto[ 0 ] = '\0';
42     while ( fgets( linha, sizeof( linha ), stdin ) !=
43            NULL ) {
44         if ( strncmp( linha, "END", 3 ) == 0 ) break;
45         if ( strlen( texto ) + strlen( linha ) <
46            MAX_TEXTO - 1 ) {
47             strcat( texto, linha );
48         }
49     }
50
51     /* Aplica todas as substituicoes */
52     for ( i = 0; i < n; ++i ) {
53         substituirTodas( texto, subs[ i ].generificado
54            , subs[ i ].neutro );
55     }
56
57     printf( "\n=== Texto neutralizado ===\n%s\n",
58            texto );
59
60     return 0;
61 }
62
63 void substituirTodas( char *texto, const char *de,
64     const char *para )
65 {
66     char buffer[ MAX_TEXTO ];
67     char *p;
68     int tamDe = strlen( de );
69
70     while ( ( p = strstr( texto, de ) ) != NULL ) {
71         int prefixo = p - texto;
72         strncpy( buffer, texto, prefixo );
73         buffer[ prefixo ] = '\0';
74         strcat( buffer, para );
75         strcat( buffer, p + tamDe );
76         strcpy( texto, buffer );
77     }
78 }
```

Reflexão

Atenção

A neutralidade linguística em português é um tema mais complexo do que em inglês por causa da *flexão de gênero em substantivos e adjetivos*. Soluções profissionais consideram:

- **Reescrita estrutural:** “o aluno deve fazer” → “é necessário fazer”
- **Formas duplas:** “aluno/aluna”, “aluno(a)”
- **Linguagem inclusiva moderna:** “alunes”, “alunx” (controverso)
- **Coletivos:** “estudantes”, “pessoa interessada”

A substituição automática é apenas o primeiro passo. Sistemas avançados usam processamento de linguagem natural (PLN) para entender contexto e propor reescritas mais naturais.

Boa prática de programação

Padrão comum nos 4 exercícios desta seção: todos envolvem *substituição contextual* em texto, e implementam o algoritmo central `substituirTodas` (busca + substituição). Reusabilidade!

Este é o princípio do **DRY** (*Don't Repeat Yourself*): identificar o padrão e implementar uma vez. O Cap. 5 (funções) e o Cap. 7 (ponteiros) deram a você as ferramentas necessárias para fazê-lo elegantemente.

Reflexão Final do Capítulo 8

A computação a serviço da linguagem humana

O Capítulo 8 nos leva a um território fascinante: a interseção entre programação e linguagem natural. Os exercícios “Fazendo a Diferença” demonstram aplicações de manipulação de strings que tocam questões sociais profundas:

- **Saúde pública** (8.43) – ajudar pessoas a comer melhor.
- **Segurança digital** (8.44) – proteger usuários de e-mails maliciosos.
- **Comunicação** (8.45) – traduzir entre dialetos da era digital.
- **Inclusão social** (8.46) – promover linguagem mais inclusiva.

Em todos eles, o ponto central é o mesmo: **texto é uma forma de dados extremamente importante**, e dominar suas manipulações abre portas para aplicações com impacto real na sociedade.

Aonde isso leva? O processamento de linguagem natural (PLN) é hoje uma das áreas mais quentes da computação. Modelos de linguagem como GPT, BERT e Claude são, no fundo, sistemas extraordinariamente sofisticados de manipulação de strings – com bilhões de parâmetros, mas baseados em ideias fundamentais que você acabou de praticar: tokenização, busca, substituição, classificação.

Boa prática de programação

Próximos passos: no Capítulo 9, estudaremos formatação detalhada de E/S; no Capítulo 10, estruturas (**struct**) que permitem agrupar dados heterogêneos; no Capítulo 11, manipulação de arquivos (essencial para processamento de textos do mundo real); no Capítulo 12, estruturas de dados encadeadas (que suportam dicionários, tabelas hash, parsing de árvores sintáticas, etc.). O que você acabou de aprender é a base de tudo.